

User Manual

Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 - Machine Learning | Assignment 1

Department of Computer Science and Engineering

1. Prerequisites

To run this project, you need:

- Python 3.8 or higher
- No external libraries required (uses only Python standard library)

To verify your Python version:

```
python3 --version
```

2. Project Structure

```
ML_Project/  
  structures.py    - Data classes (State, Transition, etc.)  
  planner.py      - A* planner with safety and scoring  
  test_cases.py   - All 6 test case definitions  
  main.py         - Main runner script  
  README.md       - Project documentation  
  docs/           - PDF deliverables
```

3. Running the Project

To run all 6 test cases and see the results:

```
cd ML_Project  
python3 main.py
```

This will execute all test cases including dynamic replanning scenarios and print detailed metrics for each one.

4. Understanding the Output

For each test case, the output shows:

- Success: Whether a valid path was found (True/False)
- State Path: The ordered list of state IDs from start to goal
- Transition Path: The ordered list of transition IDs used
- Total Cost: Sum of transition costs along the path
- Safety Score: Minimum Euclidean distance from any path state to nearest bad state
- Reliability: Sum of reliability values of transitions used
- Objective Score: Combined score using $\text{Score}(P) = aG - bC + gD + dR$
- Explored States: Number of states the algorithm examined

- Planning Time: Time taken to compute the path (in seconds)

5. Creating Custom Test Cases

You can create your own test cases by following these steps:

Step 1: Define States

```
from structures import State, Transition, PlanningProblem

# Create states with IDs and 2D positions
s = State(id=0, embedding=[0.0, 0.0]) # Start
a = State(id=1, embedding=[1.0, 0.0]) # Intermediate
g = State(id=2, embedding=[2.0, 0.0]) # Goal
```

Step 2: Define Transitions

```
# Create transitions between states
t1 = Transition(
    id=1,
    from_state=0, to_state=1,
    cost=1.0, safety=1.0,
    reliability=1.0, available=True
)
t2 = Transition(
    id=2,
    from_state=1, to_state=2,
    cost=1.0, safety=1.0,
    reliability=1.0, available=True
)
```

Step 3: Create Problem and Run

```
from planner import SafePlanner

problem = PlanningProblem(
    initialState=0,
    goalState=2,
    badStates=[],
    states=[s, a, g],
    transitions=[t1, t2]
)

planner = SafePlanner()
result = planner.plan(problem)
print(result)
```

6. Dynamic Replanning

The planner provides three replanning functions:

6.1 Disable a Transition

```
from planner import replan_with_unavailable_transition

# Make transition from state 1 to state 2 unavailable
result = replan_with_unavailable_transition(
    planner, problem, from_state=1, to_state=2
)
```

6.2 Change the Goal

```
from planner import replan_with_new_goal

# Change goal to state 3
result = replan_with_new_goal(
    planner, problem, new_goal=3
)
```

6.3 Add a New Transition

```
from planner import replan_with_new_transition
from structures import Transition

# Add a shortcut transition
shortcut = Transition(
    id=99, from_state=0, to_state=2,
    cost=0.5, safety=1.0,
    reliability=1.0, available=True
)
result = replan_with_new_transition(
    planner, problem, shortcut
)
```

7. Tuning the Scoring Function

The scoring function $\text{Score}(P) = \alpha \cdot G - \beta \cdot C + \gamma \cdot D + \delta \cdot R$ uses default weights. You can change them in `planner.py` inside the `compute_score()` function:

- Increase β to penalize expensive paths more heavily.
- Increase γ to reward paths that stay far from bad states.
- Increase δ to favor more reliable transitions.
- Increase α to strongly reward reaching the goal.